ECEN/CSEN 123 Lab 4 Report

Tuesday 2:15 - 5:00

Austin Petersen, Dylan Thornburg, Madeline Follosco

I. Objectives

- Connect up the motors and motor drivers
- Operate the motors in a variety of ways
- Use PWM to generate speed controls

III. Lab Procedure 1

Experiment 1:

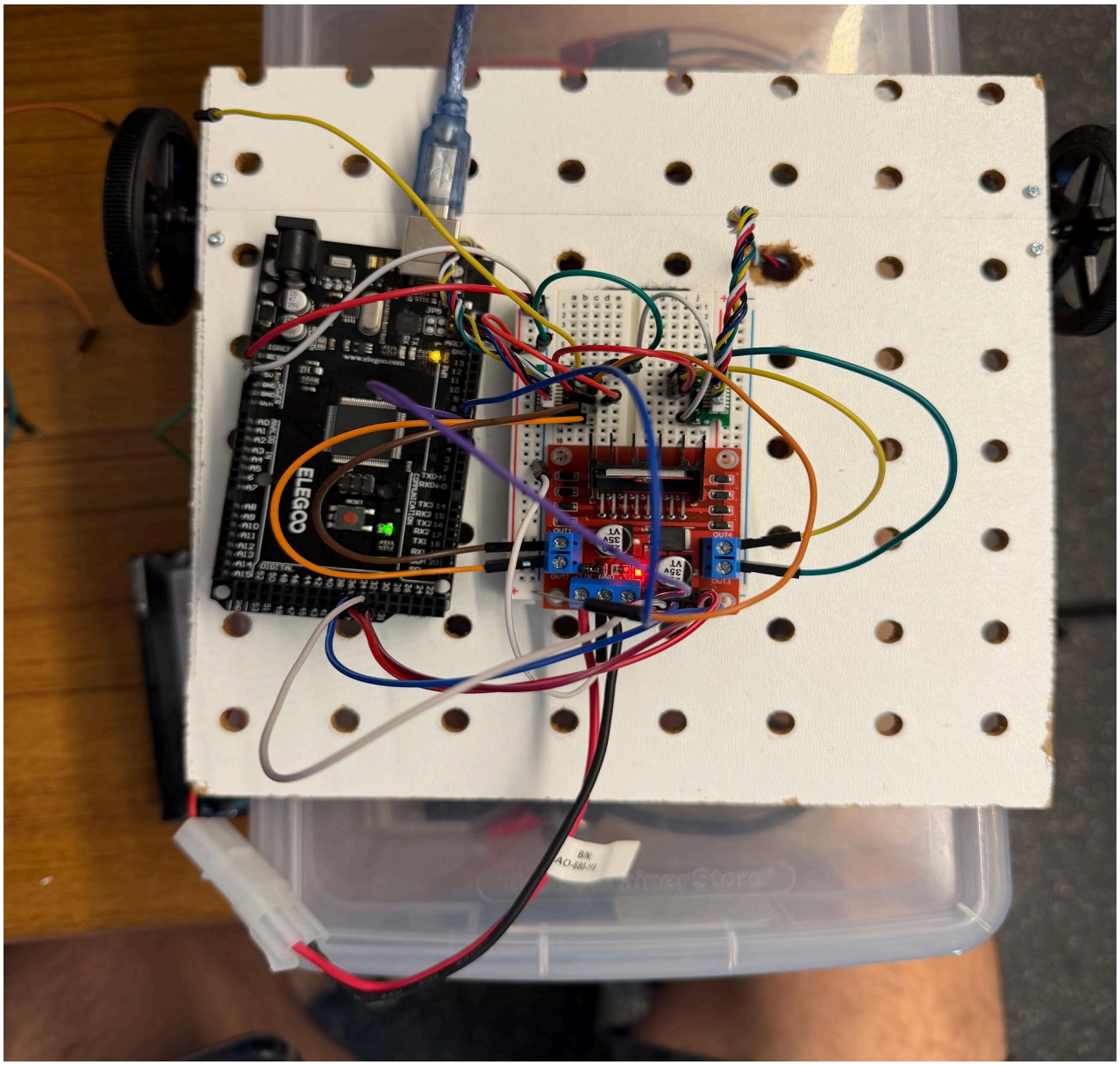
Choose output pins on your Arduino to generate the three motor control signals for each motor. Write 8 functions for your Arduino with the following names:

Forward()
Reverse()
Brake()
Coast()
TurnLeft()
TurnRight()
PivotLeft()
PivotRight()

You should be able to figure out from the name what each one is supposed to do. Each function is of type void and requires no parameters. Calling the routine makes the platform begin to move in the named way then returns to the calling program.

It is important that when switching from one function to another, you do not inadvertently brake the motor.

Create an interactive main program that effectively demonstrates that each of these routines work properly. For example, you can type a number between 0 and 7 on the serial console to select each of these. (Demo #1)



We chose 1-8 instead of 0-7.

CODE

`#include <TimerOne.h>`
`#include <math.h>`
 	
`#define PIN_CHANNEL_A 2`
`#define PIN_CHANNEL_B 3`

```
#define COUNTS_PER_REV 1800 //900 measured from previous, but now we
double count
                            // with both rising and falling edge
checking
#define PIN_EN_A 6 //change to whatever
#define PIN_IN1 32
#define PIN_IN2 34

#define PIN_EN_B 7 //change to whatever
#define PIN_IN3 33
#define PIN_IN4 35

volatile int flag = 0;
volatile long count = 0;
volatile int timerCount = 0;
long recentCount = 0;


void setup() {
  Serial.begin(9600);
  Serial.println("Setup complete.");
  //Commands to control motors
  Serial.println("1=coast, 2=forward, 3=reverse, 4=right, 5=left,
6=pivot right, 7=pivot left, 8=brake.");
  //lab recommends 0-7 but that is obnoxious on a keyboard
  pinMode(PIN_CHANNEL_A, INPUT_PULLUP);
  pinMode(PIN_CHANNEL_B, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_CHANNEL_A), isr0,
CHANGE); //any change

  pinMode(PIN_EN_A, OUTPUT);
  pinMode(PIN_IN1, OUTPUT);
  pinMode(PIN_IN2, OUTPUT);
  pinMode(PIN_EN_B, OUTPUT);
  pinMode(PIN_IN3, OUTPUT);
  pinMode(PIN_IN4, OUTPUT);

  coast(); //default coast
```

```
}


void loop() {
  if(flag) {
    flag=0;
    long temp = count;
    if(temp != recentCount){
      recentCount=temp;
      float distance = float(recentCount) * (PI * 60.0f) / (float)
COUNTS_PER_REV;
      //Serial.println("The distance is " + String(distance) + "mm");
    }
  }
  if(Serial.available() > 0) {
    int x = Serial.read(); //has ascii val for num
    x -= '0';
    if(x > 0 && x < 9) {
      if(x==1) coast();
      else if(x==2) forward();
      else if(x==3) reverse();
      else if(x==4) right();
      else if(x==5) left();
      else if(x==6) pivotRight();
      else if(x==7) pivotLeft();
      else if(x==8) brake();
    }
  }
}

void isr0() {
  int a = digitalRead(PIN_CHANNEL_A);
  int b = digitalRead(PIN_CHANNEL_B);
  if (a==b) count++;
  else count--;
  flag=1;
}
```

```
//For movement functions, enable pins should go to 0 to make the
transition smooth
void forward(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, HIGH);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, HIGH);

  analogWrite(PIN_EN_A, 255);
  analogWrite(PIN_EN_B, 255);
}

void reverse(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, HIGH);
  digitalWrite(PIN_IN3, HIGH);
  digitalWrite(PIN_IN4, LOW);

  analogWrite(PIN_EN_A, 255);
  analogWrite(PIN_EN_B, 255);
}

void coast(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);
  /* Doesn't matter but for power efficiency, set all to LOW*/
  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, LOW);
}
```

```
void right(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, HIGH);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, LOW);

  analogWrite(PIN_EN_A, 255);
  //analogWrite(PIN_EN_B, 255);
}

void left(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, HIGH);

  //analogWrite(PIN_EN_A, 255);
  analogWrite(PIN_EN_B, 255);
}

void pivotRight(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, HIGH);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, HIGH);
  digitalWrite(PIN_IN4, LOW);

  analogWrite(PIN_EN_A, 255);
  analogWrite(PIN_EN_B, 255);
}
```

```
void pivotLeft(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, HIGH);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, HIGH);

  analogWrite(PIN_EN_A, 255);
  analogWrite(PIN_EN_B, 255);
}

void brake(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  //ALL low to conserve power (may have to change to high tho)
  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, LOW);

  analogWrite(PIN_EN_A, 255);
  analogWrite(PIN_EN_B, 255);
}
```

Experiment 2:

Now add additional routines:

cmForward (int x)
cmReverse (int x)

When you call each routine, the robot will move x cm in the specified direction then return from the routine. Demonstrate these to yourself with a main program with several examples. Document for your report.

The code worked as intended.

CODE

```
#include <TimerOne.h>
#include <math.h>

#define PIN_CHANNEL_A 2
#define PIN_CHANNEL_B 3
#define COUNTS_PER_REV 1540 //changed from 1800 as now we are faster
#define PIN_EN_A 6 //change to whatever
#define PIN_IN1 32
#define PIN_IN2 34

#define PIN_EN_B 7 //change to whatever
#define PIN_IN3 33
#define PIN_IN4 35

volatile int flag = 0;
volatile long count = 0;
volatile int timerCount = 0;
long recentCount = 0;


void setup() {
  Serial.begin(9600);
  Serial.println("Setup complete.");
  //Commands to control motors
  Serial.println("1=coast, 2=forward, 3=reverse, 4=right, 5=left,
6=pivot right, 7=pivot left, 8=brake, 9=cmForward, 0=cmReverse.");
  //lab recommends 0-7 but that is obnoxious on a keyboard
  pinMode(PIN_CHANNEL_A, INPUT_PULLUP);
  pinMode(PIN_CHANNEL_B, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_CHANNEL_A), isr0,
CHANGE); //any change

  pinMode(PIN_EN_A, OUTPUT);
```

```
  pinMode(PIN_IN1, OUTPUT);
  pinMode(PIN_IN2, OUTPUT);
  pinMode(PIN_EN_B, OUTPUT);
  pinMode(PIN_IN3, OUTPUT);
  pinMode(PIN_IN4, OUTPUT);

  coast(); //default coast
}



void loop() {
  if(flag) {
    flag=0;
    long temp = count;
    if(temp != recentCount){
      recentCount=temp;
      float distance = float(recentCount) * (PI * 60.0f) / (float)
COUNTS_PER_REV;
      //Serial.println("The distance is " + String(distance) + "mm");
    }
  }
  if(Serial.available() > 0) {
    int x = Serial.read(); //has ascii val for num
    x -= '0';
    int dist = 20;
    if(x >= 0 && x <= 9) {
      if(x==1) coast();
      else if(x==2) forward();
      else if(x==3) reverse();
      else if(x==4) right();
      else if(x==5) left();
      else if(x==6) pivotRight();
      else if(x==7) pivotLeft();
      else if(x==8) brake();
      else if(x==9) cmForward(dist);
      else if(x==0) cmReverse(dist);
    }
  }
```

```
}

void isr0() {
  int a = digitalRead(PIN_CHANNEL_A);
  int b = digitalRead(PIN_CHANNEL_B);
  if (a==b) count++;
  else count--;
  flag=1;
}

//For movement functions, enable pins should go to 0 to make the
transition smooth
void forward(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, HIGH);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, HIGH);

  analogWrite(PIN_EN_A, 255);
  analogWrite(PIN_EN_B, 255);
}

void reverse(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, HIGH);
  digitalWrite(PIN_IN3, HIGH);
  digitalWrite(PIN_IN4, LOW);

  analogWrite(PIN_EN_A, 255);
  analogWrite(PIN_EN_B, 255);
}
```

```
void coast(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);
  /* Doesn't matter but for power efficiency, set all to LOW*/
  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, LOW);
}

void right(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, HIGH);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, LOW);

  analogWrite(PIN_EN_A, 255);
  //analogWrite(PIN_EN_B, 255);
}

void left(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, HIGH);

  //analogWrite(PIN_EN_A, 255);
  analogWrite(PIN_EN_B, 255);
}

void pivotRight(){
  analogWrite(PIN_EN_A, 0);
```

```
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, HIGH);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, HIGH);
  digitalWrite(PIN_IN4, LOW);

  analogWrite(PIN_EN_A, 255);
  analogWrite(PIN_EN_B, 255);
}

void pivotLeft(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, HIGH);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, HIGH);

  analogWrite(PIN_EN_A, 255);
  analogWrite(PIN_EN_B, 255);
}

void brake(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  //ALL low to conserve power (may have to change to high tho)
  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, LOW);

  analogWrite(PIN_EN_A, 255);
  analogWrite(PIN_EN_B, 255);
}
```

```
float COUNTS_PER_CM = (float)COUNTS_PER_REV / (PI * 6.0f);

void cmForward(int dist){
  long start = count;
  long goal = (long) round(dist * COUNTS_PER_CM);
  forward();
  while(labs(count-start) < goal) {}
  brake();
}

void cmReverse(int dist){
  long start = count;
  long goal = (long) round(dist * COUNTS_PER_CM);
  reverse();
  while(labs(count-start) < goal) {}
  brake();
}
```

Experiment 3:

Speed Control:

Connect DIP switches to your Arduino with an appropriate circuit so that you can select at least 4 different speeds without reprogramming the Arduino.

Write a program that uses PWM to drive the motors forward at:

a. 25% duty cycle

b. 40% duty cycle

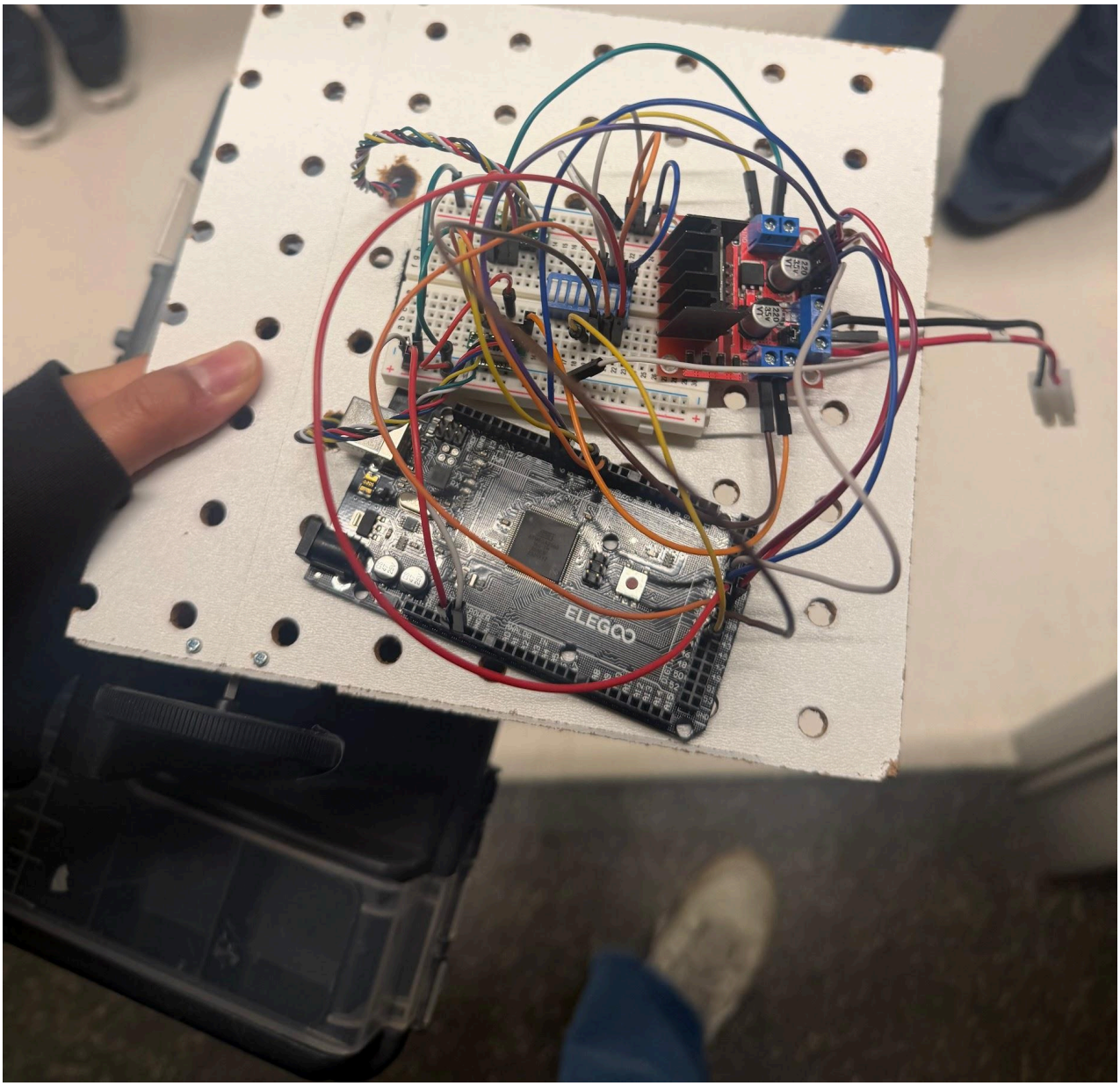
c. 70% duty cycle

d. Full Speed (100% duty cycle)

based on the switch settings.

Using your wheel sensors and timing functions, the program should (after a reset) drive forward at the speed on the switches for 1 second, then measure the distance travelled for the next 2 seconds, then stop. Calculate the speed and print it on the serial console. Document these speeds in your lab report. Explain

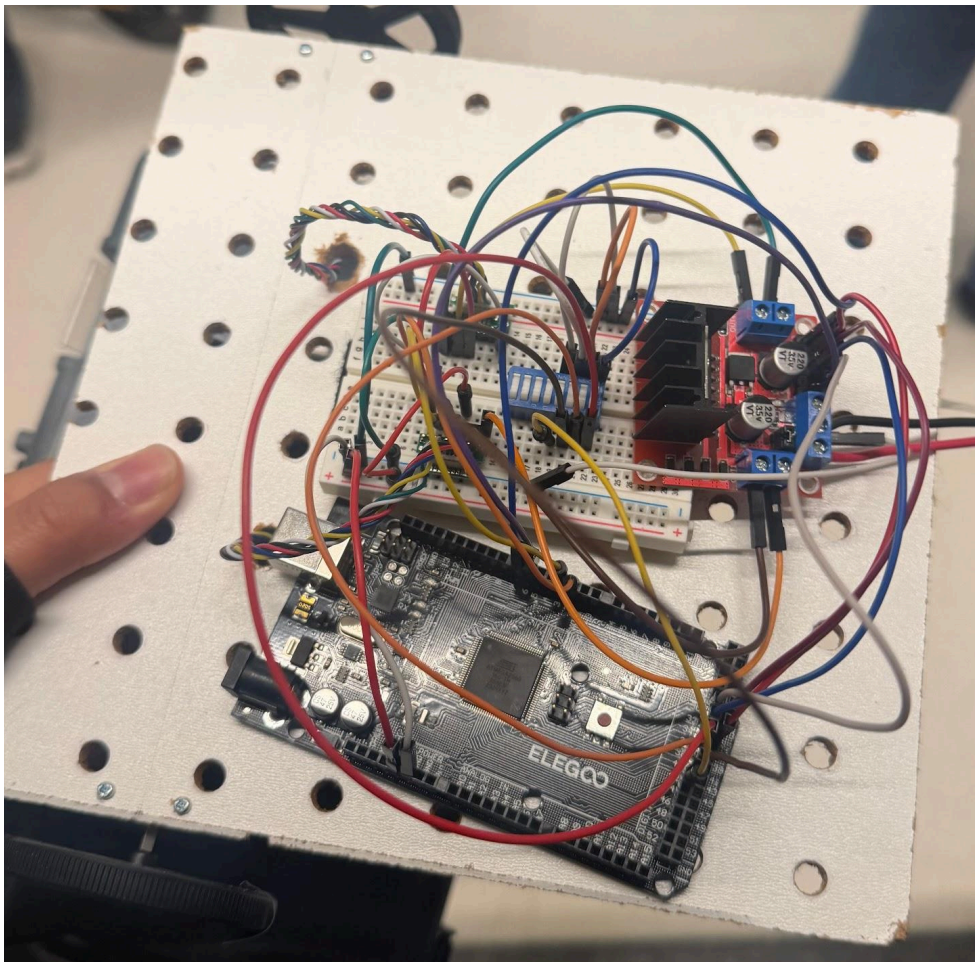
any inaccuracies.



```
PWM=64
Distance: 58.20 cm
Speed: 29.10 cm/s
PWM=102
Distance: 64.90 cm
Speed: 32.45 cm/s
PWM=179
Distance: 72.52 cm
Speed: 36.26 cm/s
PWM=255
Distance: 74.67 cm
Speed: 37.33 cm/s
```

Observations: For every duty cycle the number of interrupts was the same. However, we noticed around 10% error at worst (lowest speed), and below 1% error at best (highest speed) in distance and velocity. We attribute this to acceleration and deceleration with the initial start and stop of the wheels.

(Demo #2) – show whichever speeds the TA selects



CODE

```
#include <TimerOne.h>
#include <math.h>

#define PIN_CHANNEL_A 2
#define PIN_CHANNEL_B 3
#define COUNTS_PER_REV 1600 //changed from 1800 as now we are faster
#define PIN_EN_A 6 //change to whatever
#define PIN_IN1 32
#define PIN_IN2 34

#define PIN_EN_B 7 //change to whatever
#define PIN_IN3 33
```

```
#define PIN_IN4 35

volatile int flag = 0;
volatile long count = 0;
volatile int timerCount = 0;
long recentCount = 0;
int switchPin[4] = {36,38,40,42};
int pwm=255; //def 255
int j;
void setup() {
  Serial.begin(9600);
  Serial.println("Setup complete.");
  //Commands to control motors
  Serial.println("1=coast, 2=forward, 3=reverse, 4=right, 5=left,
6=pivot right, 7=pivot left, 8=brake, 9=cmForward, 0=cmReverse, '-'
to run .");
  //lab recommends 0-7 but that is obnoxious on a keyboard
  pinMode(PIN_CHANNEL_A, INPUT_PULLUP);
  pinMode(PIN_CHANNEL_B, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_CHANNEL_A), isr0,
CHANGE); //any change
  Timer1.initialize(100000);// 10Hz
  Timer1.attachInterrupt(timerIsr);

  pinMode(PIN_EN_A, OUTPUT);
  pinMode(PIN_IN1, OUTPUT);
  pinMode(PIN_IN2, OUTPUT);
  pinMode(PIN_EN_B, OUTPUT);
  pinMode(PIN_IN3, OUTPUT);
  pinMode(PIN_IN4, OUTPUT);

  coast(); //default coast

  for(j=0; j < 4; j++) pinMode(switchPin[j], INPUT_PULLUP);
}


void loop() {
```

```
  if(flag) {
    flag=0;
    long temp = count;
    if(temp != recentCount){
      recentCount=temp;
      float distance = float(recentCount) * (PI * 60.0f) / (float)
COUNTS_PER_REV;
      //Serial.println("The distance is " + String(distance) + "mm");
    }
  }
  if(Serial.available() > 0) {
    int x = Serial.read(); //has ascii val for num
    x -= '0';
    int dist = 19;
    if(x >= 0 && x <= 9) {
      if(x==1) coast();
      else if(x==2) forward();
      else if(x==3) reverse();
      else if(x==4) right();
      else if(x==5) left();
      else if(x==6) pivotRight();
      else if(x==7) pivotLeft();
      else if(x==8) brake();
      else if(x==9) cmForward(dist);
      else if(x==0) cmReverse(dist);
    }
    else if(x==('-' - '0')) speedTest();
  }
}

void isr0() {
  int a = digitalRead(PIN_CHANNEL_A);
  int b = digitalRead(PIN_CHANNEL_B);
  if (a==b) count++;
  else count--;
  flag=1;
}
```

```
//For movement functions, enable pins should go to 0 to make the
transition smooth
void forward(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, HIGH);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, HIGH);

  analogWrite(PIN_EN_A, pwm);
  analogWrite(PIN_EN_B, pwm);
}

void reverse(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, HIGH);
  digitalWrite(PIN_IN3, HIGH);
  digitalWrite(PIN_IN4, LOW);

  analogWrite(PIN_EN_A, pwm);
  analogWrite(PIN_EN_B, pwm);
}

void coast(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);
  /* Doesn't matter but for power efficiency, set all to LOW*/
  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, LOW);
}
```

```
void right(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, HIGH);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, LOW);

  analogWrite(PIN_EN_A, pwm);
  //analogWrite(PIN_EN_B, 255);
}

void left(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, HIGH);

  //analogWrite(PIN_EN_A, 255);
  analogWrite(PIN_EN_B, pwm);
}

void pivotRight(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, HIGH);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, HIGH);
  digitalWrite(PIN_IN4, LOW);

  analogWrite(PIN_EN_A, pwm);
  analogWrite(PIN_EN_B, pwm);
}
```

```
void pivotLeft(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, HIGH);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, HIGH);

  analogWrite(PIN_EN_A, pwm);
  analogWrite(PIN_EN_B, pwm);
}

void brake(){
  analogWrite(PIN_EN_A, 0);
  analogWrite(PIN_EN_B, 0);

  //ALL low to conserve power (may have to change to high tho)
  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, LOW);

  analogWrite(PIN_EN_A, pwm);
  analogWrite(PIN_EN_B, pwm);
}

float COUNTS_PER_CM = (float)COUNTS_PER_REV / (PI * 6.0f);

void cmForward(int dist){
  long start = count;
  long goal = (long) round(dist * COUNTS_PER_CM);
  forward();
  while(labs(count-start) < goal) {}
  brake();
  Serial.println(String(goal));
}
```

```
void cmReverse(int dist){
  long start = count;
  long goal = (long) round(dist * COUNTS_PER_CM);
  reverse();
  while(labs(count-start) < goal) {}
  brake();
}



int readSpeed(){
  if(digitalRead(switchPin[0]) == LOW) return(int)round(255.0f *
.25f);
  else if(digitalRead(switchPin[1]) == LOW) return(int)round(255.0f *
.4f);
  else if(digitalRead(switchPin[2]) == LOW) return(int)round(255.0f *
.7f);
  else if(digitalRead(switchPin[3]) == LOW) return 255;
  else return 0; //no switch on
}

void speedTest(){
  pwm = readSpeed();
  Serial.println("PWM=" + String(pwm));

  coast();
  delay(10);

  forward();
  delay(1000);

  long startMeas = count;
  delay(2000);
  long endMeas = count;

  brake();
```

```
  long countsMeas = labs(endMeas - startMeas);
  float distance = ((float)countsMeas * (PI * 60.0f) /
((float)COUNTS_PER_REV * 10.0f));
  float speed = distance/ 2.0f;

  //Serial.println()
  Serial.println("Distance: " + String(distance) + " cm");
  Serial.println("Speed: " + String(speed) + " cm/s");
}

void timerIsr() {
  pwm=pwm = readSpeed();
}
```